

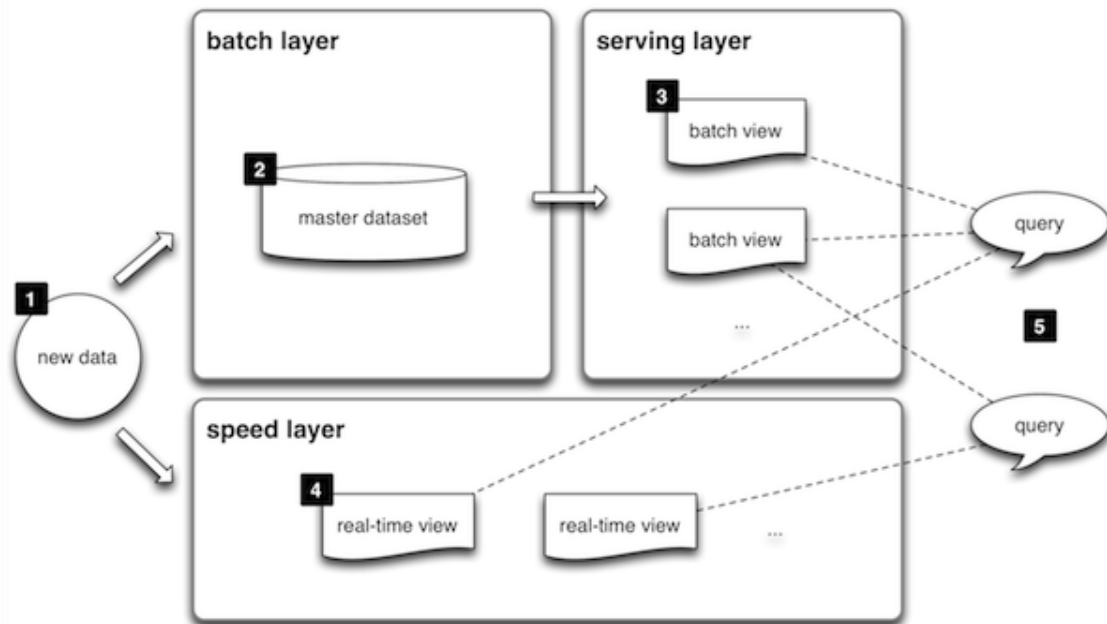
TD7: Ingeniería de Datos

Arquitectura Lambda.
Almacenamiento

Speed Layer

Serving Layer

Batch Layer



Llegamos a algunas conclusiones



File system distribuido como infra de almacenamiento

Frameworks de serialización

Llegamos a algunas conclusiones

- Los **serialization frameworks** proporcionan herramientas y bibliotecas para usar objetos de distintos lenguajes.
- Pueden serializar un objeto en un array de bytes desde cualquier lenguaje, y luego deserializar ese objeto en cualquier lenguaje.
- Proporcionan un lenguaje de definición de esquemas para definir objetos y sus campos, también mecanismos para versionar objetos de forma segura para que un esquema pueda evolucionar sin invalidar las versiones previas.
- Los tres **serialization frameworks** notables son **Thrift**, **Protocol Buffers** y **Avro**.

Arquitectura Lambda – Formatos de Almacenamiento

Formatos de almacenamiento - Avro

— — —



Formato **Avro**

- ❖ Permite almacenar el **esquema** junto con el mensaje
 - El esquema se define en JSON
- ❖ Aporta **compresión** de los datos
- ❖ Organiza la información en filas: row oriented
 - Optimizado para leer **filas completas**
 - Optimizado para leer muchas **filas consecutivas**

Formatos de almacenamiento - Avro



- ❖ Tiene soporte para **estructuras complejas** (como registros anidados, arrays, maps, etc.)
- ❖ Provee **bibliotecas** en los principales lenguajes para poder serializar y deserializar objetos
- ❖ Organiza los registros en **bloques de tamaño fijo**.
 - Muy importante para usar en un filesystem distribuido

Formatos de almacenamiento - Avro



Formato **Avro** - Definición de esquema

```
{
  "type": "record",
  "name": "UserInteraction",
  "fields": [
    {"name": "user-id", "type": "int"},
    {"name": "content", "type": ["null", "string"], "default": null},
    {"name": "type", "type": {
      "type": "enum",
      "name": "InteractionType",
      "symbols": [
        "PLAY",
        "LIKE"
      ]
    }},
    {"name": "timestamp", "type": "datetime"}
  ]
}
```


Formatos de almacenamiento - Avro



— — —

Cada archivo se organiza en dos grandes partes:

❖ Un **header** con:

- Un **magic number** de 4 bytes para identificación
- El **esquema** de serialización (en formato JSON)
- **Metadata** extra
- Un **marcador de sincronización** de 16 bytes para indicar separaciones entre bloques

❖ Los **bloques de datos**, cada uno con:

- Un indicador de **cuántos registros** contiene
- Un indicador del **tamaño en bytes** total para el bloque
- Los **objetos** serializados
- El **marcador de sincronización** al final

Formatos de almacenamiento - Parquet



Formato **Parquet**

- ❖ También aporta **compresión**:
 - La organización columnar permite mucha mayor eficiencia.
 - Establece **encoding** específico para cada columna
- ❖ También permite **particionar** los archivos (para consumo distribuido)
- ❖ Brinda optimización para **procesamiento**:
 - Sólo se leen las columnas que se necesitan (en general, no todas)
 - Soporta **predicate pushdown**: acercar la lógica de la recuperación de datos al origen de los datos, ejecutar determinadas operaciones directamente en el origen, para ahorrar tiempo y potenciar el procesamiento al no llevar todos los datos de la red al motor.

Formatos de almacenamiento - Parquet



— — —
También:

- ❖ Permite almacenar el **esquema** junto con el mensaje: No tiene lenguaje propio de definición de esquema (se pueden usar otros como el de Thrift o Avro)
- ❖ Soporta tipos complejos (tanto como los tipos de esquema que permite)
- ❖ Cuenta con bibliotecas para los principales lenguajes

Formatos de almacenamiento - Parquet



— — —

Cada archivo se organiza en cuatro partes:

- ❖ Un **magic number** de 4 bytes para identificación
- ❖ Metadata sobre los **row groups** (los registros están agrupados) incluyendo su offset dentro del archivo.
- ❖ Los **column chunks** (primero todos los del row group 1, después los del 2)
- ❖ El **footer** del archivo, incluyendo el **esquema**, información de **row groups**, e información (como encoding) y ubicación de los distintos **chunks**

Almacenamiento del Master Dataset

Almacenamiento del Master Dataset



— — —

Sabemos que queremos almacenar nuestros datos en archivos por sus ventajas inherentes (append only, inmutabilidad, etc):

- ❖ Para esto necesitamos un sistema de manejo de archivos, pero los **file systems** tradicionales no garantizan otras necesidades como la escalabilidad o la capacidad de procesamiento en paralelo
- ❖ Necesitamos definir cómo vamos a organizar esos archivos:
 - ¿Un archivo para todos los datos?
 - ¿Muchos archivos pequeños con grupos aleatorios de eventos?
 - ¿Carpetas para agruparlos de alguna manera?

Almacenamiento distribuido

Almacenamiento - Master Dataset



File system distribuido

- ❖ Estructura y categorías de los file systems tradicionales, en el contexto de un **sistema distribuido**
- ❖ **Múltiples soportes físicos** detrás de un único file system lógico.

Proveen muchas garantías no funcionales:

- Escalabilidad (manejo de grandes volúmenes)
- High availability
- Paralelización de lecturas
- Redundancia



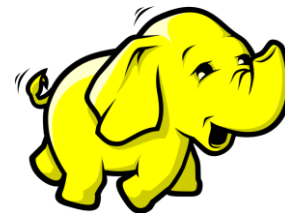
Apache Hadoop

The Apache® Hadoop® project develops open-source software for reliable, scalable, distributed computing.

The Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models. It is designed to scale up from single servers to thousands of machines, each offering local computation and storage. Rather than rely on hardware to deliver high-availability, the library itself is designed to detect and handle failures at the application layer, so delivering a highly-available service on top of a cluster of computers, each of which may be prone to failures.

Hadoop - Características

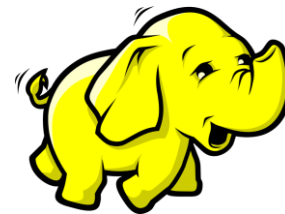
— — —



- ❖ permite a las aplicaciones hacer uso de miles de nodos de procesamiento y almacenamiento y petabytes de datos.
- ❖ principal ventaja: diseñado para ejecutar sobre hardware sencillo, normal y corriente.
- ❖ se trata de **acercar el procesamiento** al lugar en donde se encuentran almacenados los datos y así aprovechar técnicas de paralelización, aumentando la escalabilidad.
- ❖ mover el procesamiento es mucho más rápido, fácil y eficiente que mover grandes cantidades de datos, que pueden producir altas latencias y congestión en la red.

Hadoop - Características

— — —



Tres componentes principales

- ❖ HDFS (Hadoop Distributed File System)
- ❖ MapReduce
- ❖ Yarn (Yet Another Resource Negotiator)

File systems distribuidos



HDFS (Hadoop File System)

- Filesystem diseñado especialmente para **Hadoop**
- **Open source**: desarrollado en Java
- Manejo de **grandes volúmenes** en forma distribuida
 - Optimizado para **archivos muy grandes**



File systems distribuidos - HDFS

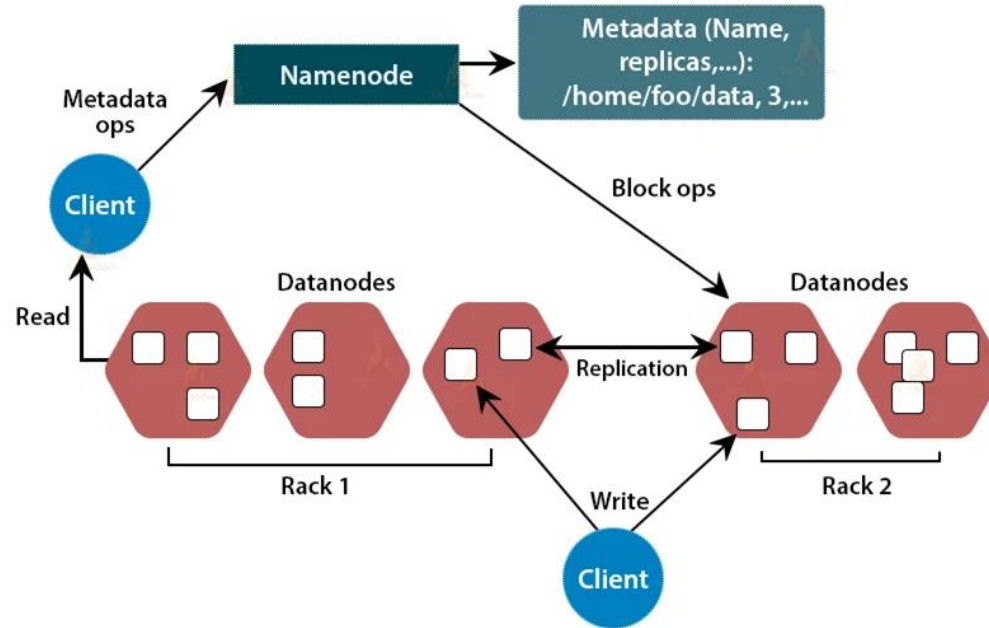


HDFS (Hadoop File System)

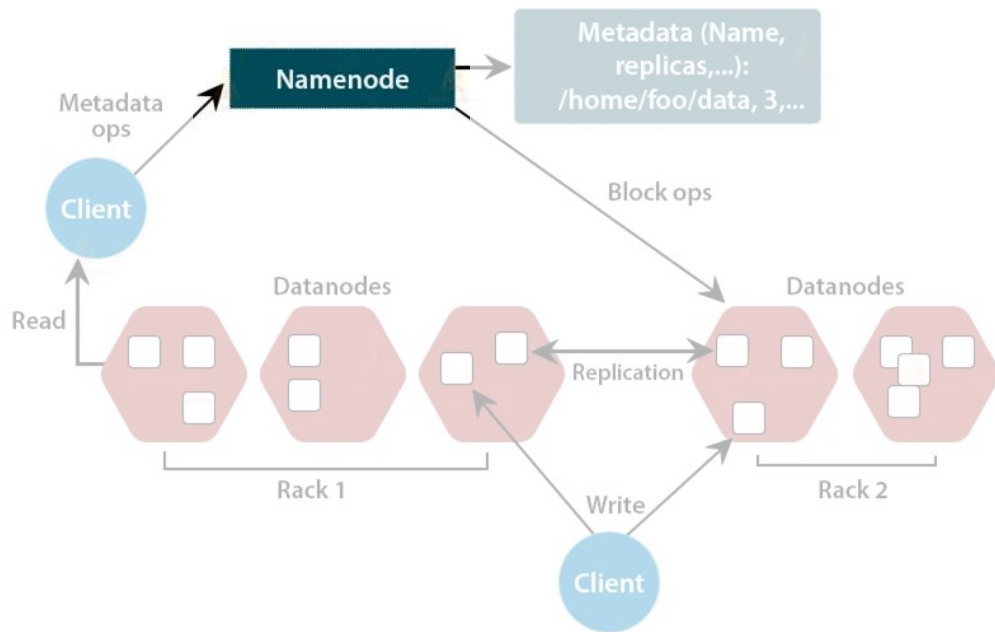
- **Cluster:** conjunto de **nodos** coordinados
- **Namenode:** nodo central (único) de **metadata del FS**
 - Puede mantener el estado en memoria
- **Datanode:** nodos **escalables** que almacenan los datos del FS
- **Archivo:** artefacto lógico a almacenar distribuidamente en Datanodes
- **Bloque:** fragmento de un archivo, de tamaño fijo, replicado en Datanodes



File systems distribuidos - HDFS

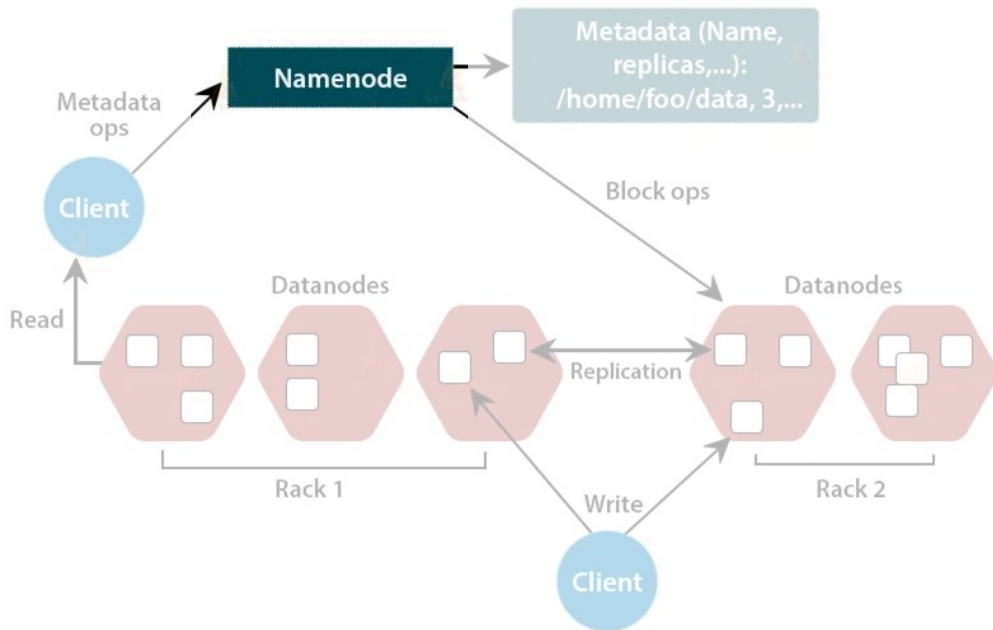


File systems distribuidos - HDFS



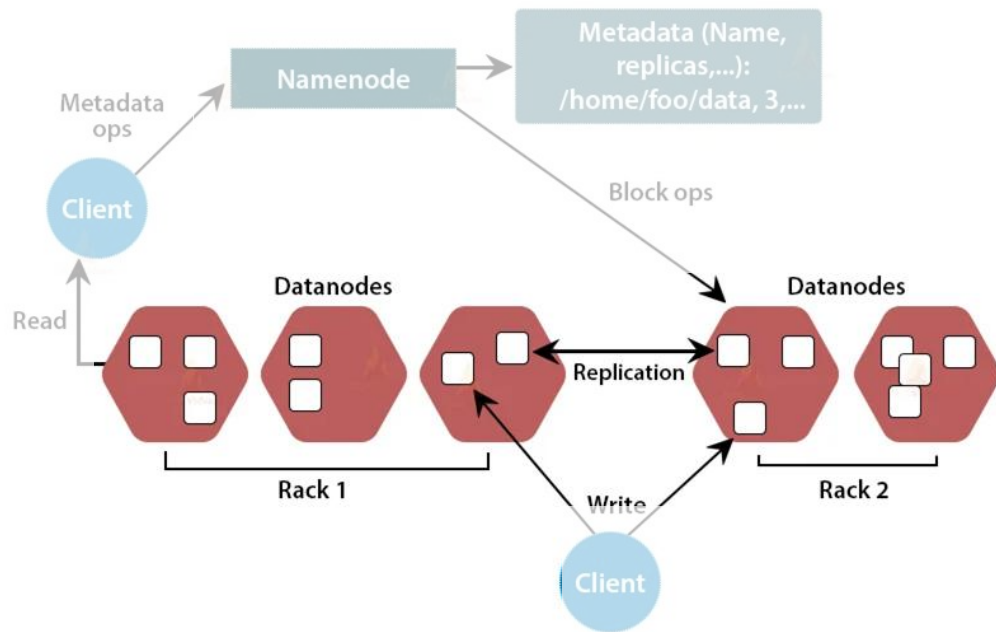
NameNode es el controlador y administrador de HDFS; es quien conoce el estado y los metadatos de todos los archivos HDFS, incluidos los permisos de archivos, los nombres y las ubicaciones de los bloques.

File systems distribuidos - HDFS



El namenode también conoce los nodos de datos en los que se encuentran todos los bloques de un archivo determinado.

File systems distribuidos - HDFS

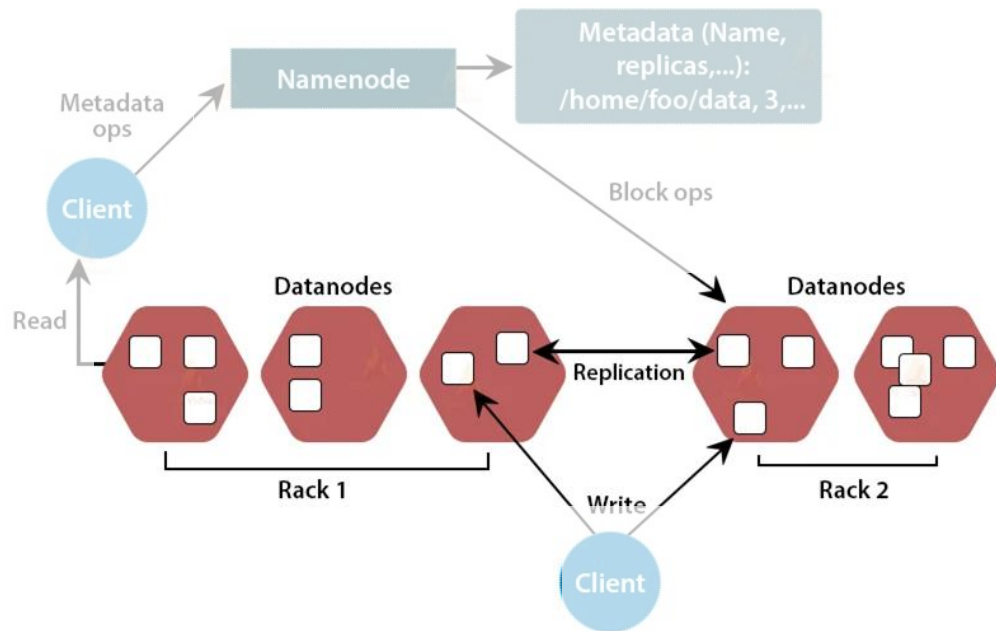


Datanodes:

Los archivos HDFS se escriben una sola vez y solo tienen un escritor en un momento dado.

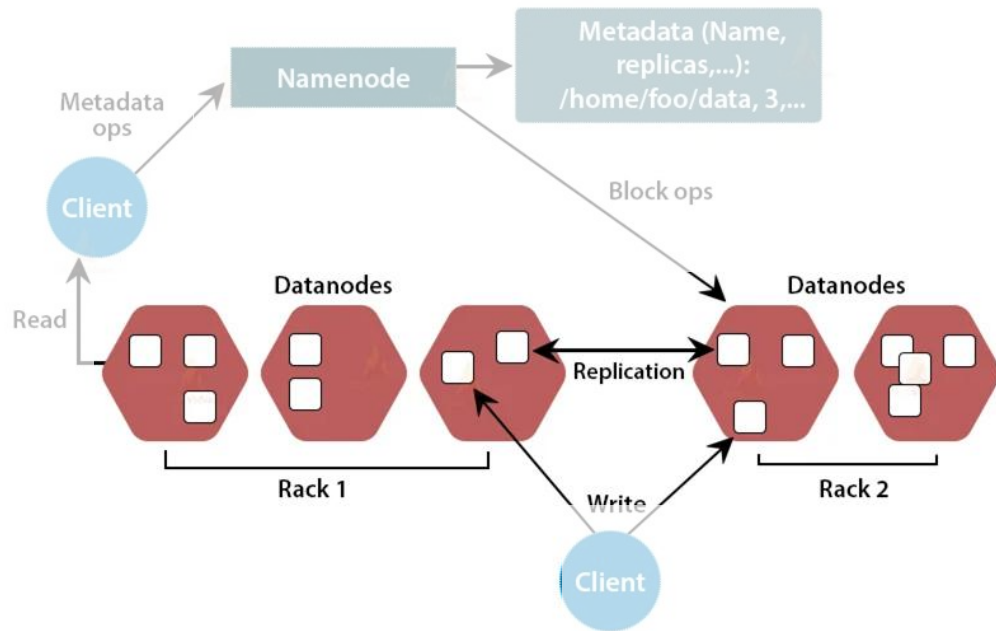
Los datanodes realizan operaciones de lectura y escritura en los sistemas de archivos, a solicitud del cliente.

File systems distribuidos - HDFS



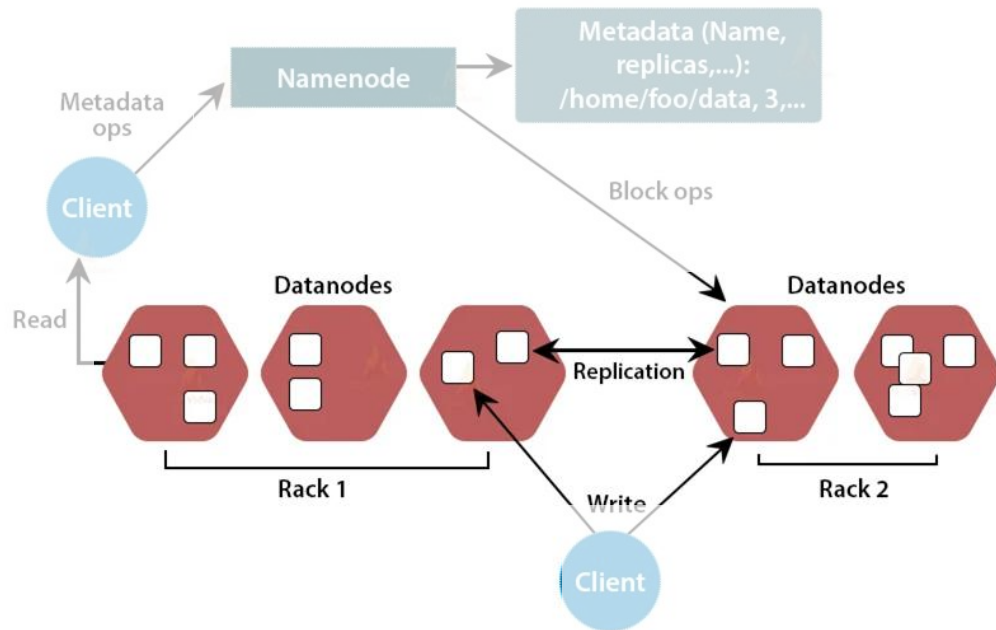
También realizan operaciones como creación, eliminación y replicación de bloques de acuerdo con las instrucciones del namenode. Almacenan y recuperan bloques cuando se les indica que lo hagan, y reportan al namenode periódicamente con listas de bloques que están almacenando.

File systems distribuidos - HDFS



Durante el funcionamiento normal, los DataNodes envían **heartbeat** al NameNode para confirmar que el DataNode está funcionando y que las réplicas de bloques que aloja están disponibles. El intervalo de latido predeterminado es de tres segundos.

File systems distribuidos - HDFS



Si el NameNode no recibe un latido de un DataNode en diez minutos, el NameNode considera que el DataNode está fuera de servicio y que las réplicas de bloques alojadas por ese DataNode no están disponibles. Luego, el NameNode programa la creación de nuevas réplicas de esos bloques en otros DataNodes.

Almacenamiento distribuido - S3



S3 (Simple Storage Service)

- ❖ Originalmente, uno de los **servicios de AWS**
- ❖ Hoy, **API (y protocolo) ampliamente adoptada**
- ❖ Permite guardar, recuperar y buscar **objetos** en filesystems distribuidos
- ❖ Basada en el concepto de **key/value store**
- ❖ Agrupa objetos en **contenedores lógicos (buckets)**

Almacenamiento distribuido



Data locality

Tanto HDFS como S3 brindan algún soporte de **Data Locality**:

- ❖ es la práctica de almacenar datos cerca de la unidad de procesamiento que los consumirá, lo que reduce el movimiento de datos y la latencia en aplicaciones con uso intensivo de datos.
- ❖ **Almacenamiento distribuido** para **procesamiento distribuido**
- ❖ Llevar el **cómputo a donde están los datos** (y no al revés)
 - **Node-local**: cuando el cómputo ocurre en el mismo nodo que tiene los datos
 - **Rack-local**: cuando al menos se garantiza el mismo rack